



# Revisão Acelerada de Programação - RAP RAP-2026, Marinha do Brasil

Centro de Instrução Almirante Alexandrino

Departamento de Aperfeiçoamento Avançado para Oficiais

Curso de Aperfeiçoamento Avançado em Guerra Acústica (C-ApA-Of-GA)

Professor: João Victor da Fonseca Pinto

Descrição do Documento: Planejamento de aulas

## Introdução

Esta disciplina de Revisão Acelerada de Programação em Python tem por finalidade consolidar os conceitos fundamentais de lógica de programação e desenvolvimento de algoritmos sob a perspectiva da Guerra Acústica. O conteúdo é estruturado de forma intensiva, visando capacitar os militares na manipulação de dados complexos, na automação de rotinas de análise e na implementação rápida de soluções computacionais aplicadas. Mediante uma abordagem prática e direcionada, os alunos serão instrumentalizados a utilizar a linguagem Python como ferramenta de suporte técnico-operacional, otimizando o processamento de fluxos de informação e garantindo maior celeridade no cumprimento das tarefas correlatas à especialidade.

## Planejamento de Aulas

- **Aula 1 (27 de julho de 2026):**

- Apresentação da matéria: Método de avaliação e cronograma.
- Conceito de Python: Entender o que é Python e por que ele é amplamente utilizado.
- Conceito de Docker: Compreender o que são contêineres e sua importância para o desenvolvimento robusto.
- Conceito de Docker Desktop: Conhecer a interface que gerencia nossos contêineres.
- Conceito de Jupyter Lab: Aprender a utilizar e navegar na interface de notebooks.
- Conceito de IDE: Aprender a utilizar o Microsoft VS Code.
- Primeiros Passos: Aprender atalhos importantes e executar seu primeiro código em Python: no terminal, em um script e em um Jupyter notebook.

- **Aula 2 (3 de agosto de 2026 e 10 de agosto de 2026):**

- Tipos e Variáveis: Entender como criar variáveis e como o Python categoriza os dados básicos.
- Casting: Aprender a converter dados de um tipo para outro.
- Built-in Functions: Explorar funções integradas como `print`, `help`, `len`, `abs`, etc.
- Tipos de Prints e Formatações: Dominar técnicas para exibir resultados formatados (como f-strings).
- Estruturas Condicionais: Tomar decisões no código usando `if`, `elif` e `else`.

- Introdução a Funções: Entender o que é uma função, para que serve e como criar as suas próprias.
- Exemplos Introdutórios: Praticar com mini-programas que unem todos os conceitos.

- **Aula 3:**

- Estruturas de Repetição: Entender os conceitos de loops (`for` e `while`).
- Repetição com `for`: Dominar a iteração sobre sequências e a utilização da função `range()` dentro de funções.
- Repetição com `while`: Compreender como repetir instruções baseadas em condições lógicas variáveis.
- Controle de Fluxo com `break` e `continue`: Aprender a interromper ou pular iterações de loops de forma inteligente.
- Exercícios Práticos: Aplicar essas técnicas em exercícios práticos.

- **Aula 4:**

- Listas em Python: Compreender o conceito de listas, declaração de listas vazias e preenchidas, e indexação positiva/negativa.
- Principais Métodos e Funções: Dominar funções como `append()`, `insert()`, `remove()`, `pop()`, `len()`, `sort()` e `reverse()`.
- Listas de Listas (Matrizes): Entender o conceito de listas aninhadas, acesso bidimensional a elementos e iteração em matrizes.
- Exercícios Práticos: Aplicar essas técnicas em exercícios práticos.

- **Aula 5:**

- Exercícios realizados em aula.

- **Aula 6:**

- Conceito de String: Entender a representação de texto, indexação, fatiamento e imutabilidade.
- Operações Básicas: Dominar concatenação, repetição, tamanho (`len()`) e operadores de pertinência (`in`).
- Métodos de Modificação: Aprender `.upper()`, `.lower()`, `.strip()` e `.replace()`.
- Divisão e Junção: Compreender `.split()` e `.join()` para converter entre texto e listas.
- Busca e Validação: Utilizar `.find()`, `.startswith()` e `.endswith()`.
- Exercícios Práticos: Aplicar essas técnicas em exercícios práticos.

- **Aula 7:**

- Exercícios realizados em aula.

- **Aula 8:**

- Conceito de Dicionário: Compreender o que é um dicionário, como chaves e valores são associados e suas propriedades básicas.
- Métodos e Funções: Dominar o uso de `.get()`, `.keys()`, `.values()`, `.items()`, `.update()`, `.pop()` e a palavra-chave `del`.
- Vantagem em Buscas: Demonstrar na prática a diferença de eficiência (complexidade de busca) entre listas e dicionários.
- Exercícios Práticos: Aplicar essas técnicas em exercícios práticos.

- **Aula 9:**

- Mutabilidade vs Imutabilidade: Compreender a diferença conceitual entre objetos que podem ser alterados na memória e aqueles que são protegidos contra escrita, especialmente ao serem passados para funções.
- Conceito de Tupla: Compreender o que é uma tupla, como ela se diferencia de listas e dicionários.
- Casos de Uso: Entender para que servem as tuplas (garantia de integridade, desempacotamento de dados e chaves de dicionários).
- Métodos de Tuplas: Conhecer a tabela de métodos específicos de tuplas (`.count()` e `.index()`).
- Exemplos e Unpacking: Dominar a sintaxe de declaração, fatiamento e desempacotamento.
- Exercícios Práticos: Aplicar essas técnicas em exercícios práticos.
- Disponibilização da primeira lista de exercícios.

- **Aula 10:**

- Entender a Abertura de Arquivos: Aprender sobre a função `open()` e seus diferentes modos (`'r'`, `'w'`, `'a'`).
- Utilizar o Gerenciador de Contexto: Aprender a melhor prática de abertura com a estrutura `with`.
- Escrever em Arquivos: Criar novos arquivos de texto e adicionar linhas com `write()` e `writelines()`.
- Ler Arquivos: Utilizar métodos de leitura como `read()`, `readline()`, `readlines()` e loops de leitura eficientes.
- Manipular Arquivos e Diretórios: Usar os módulos `os` e `pathlib` para verificar existência, renomear e excluir arquivos.
- Manipular JSON e CSV: Ler e salvar dados usando os módulos nativos do Python `json` e `csv`.

- **Aula 11:**

- Escopo de Variáveis: Compreender a diferença entre escopo local (interno à função) e escopo global (externo), além do uso consciente da palavra-chave `global`.
- Argumentos Padrão (Default): Criar funções com parâmetros pré-definidos (opcionais).
- Empacotamento de Argumentos (`*args`): Criar funções que aceitam múltiplos argumentos posicionais empacotados como uma tupla.
- Empacotamento de Argumentos Nomeados (`**kwargs`): Criar funções que aceitam múltiplos argumentos nomeados empacotados como um dicionário.
- Tipagem de Dados: Utilizar anotações de tipos (como `List`, `Dict`, `Tuple`) em assinaturas de funções para indicar os tipos esperados de entrada e retorno.
- Exercícios Práticos: Aplicar essas técnicas em exercícios práticos.

- **Aula 12:**

- Exercícios realizados em aula.

- **Aula 13:**

- Paradigma de Orientação a Objetos: Entender os conceitos de classes, objetos, atributos e métodos.
- O Método Construtor (`__init__`) e `self`: Inicializar os objetos com seus dados de forma dinâmica.
- Métodos de Instância: Definir comportamentos e manipular o estado interno de um objeto.
- Herança: Reaproveitar atributos e métodos criando classes derivadas (subclasses) a partir de uma classe base (superclass).
- Sobrescrita de Métodos (Overriding) e `super()`: Customizar funções herdadas de classes mães.
- Método Especial (`__str__`): Controlar como os nossos objetos são representados em formato de texto.
- Exercícios Práticos: Aplicar essas técnicas em exercícios práticos.

- **Aula 14:**

- Exercícios realizados em aula.

- **Aula 15:**

- Prova escrita.
- Entrega da primeira lista de exercícios.

- **Aula 16:**

- Resolução da primeira lista de exercícios.

- **Aula 17:**

- Introdução à Visualização de Dados: Compreender a importância da representação gráfica de dados e a arquitetura fundamental da biblioteca Matplotlib, discriminando os conceitos de `Figure` e `Axes`.
- Gráficos Bidimensionais (`plot` e `scatter`): Dominar a criação de gráficos de linha e de dispersão bidimensionais para a identificação de tendências e correlações em séries temporais.
- Customização de Elementos Gráficos: Personalizar componentes visuais essenciais, incluindo títulos, rótulos de eixos (`xlabel` e `ylabel`), legendas, grades, paletas de cores e estilos de marcadores.
- Histogramas e Gráficos de Barras (`hist` e `bar`): Construir representações estatísticas de distribuição de frequência e dados categóricos para análise de densidade de probabilidade de sinais.
- Múltiplos Subgráficos (`subplots`): Utilizar técnicas de particionamento de janelas gráficas para exibir múltiplos diagramas correlacionados em uma única estrutura de comparação visual.
- Exercícios Práticos: Aplicar essas técnicas em exercícios práticos.

#### • Aula 18:

- Entender o NumPy: Compreender por que o NumPy é tão rápido comparado às listas nativas do Python.
- Criar Arrays: Dominar funções de criação como `np.array()`, `np.zeros()`, `np.ones()`, `np.arange()` e `np.linspace()`.
- Inspeccionar Atributos: Entender as propriedades básicas (`.shape`, `.ndim`, `.dtype`, `.size`).
- Fatiamiento e Indexação: Acessar e recortar dados em uma e duas dimensões, incluindo o uso de máscaras booleanas.
- Vetorização: Realizar operações matemáticas rápidas elemento a elemento.
- Agregações e Eixos: Sumarizar dados utilizando `sum()`, `mean()`, `std()`, controlando o cálculo por linhas ou colunas ('axis').
- Álgebra Linear: Dominar conceitos fundamentais como produto escalar, transposição, multiplicação de matrizes, inversão e resolução de sistemas lineares ( $Ax = b$ ).

#### • Aula 19:

- Conhecer o Pandas: Entender o propósito da biblioteca e suas duas estruturas fundamentais: `Series` e `DataFrame`.
- Criar Estruturas: Aprender a inicializar `Series` e `DataFrames` a partir de dicionários e listas.
- Importar Dados (I/O): Carregar dados externos a partir de arquivos CSV.
- Inspeccionar Dados: Usar funções analíticas básicas como `.head()`, `.info()`, `.describe()` e `.shape`.

- Filtragem e Seleção: Acessar colunas, usar `.loc[]` / `.iloc[]` e aplicar filtros com máscaras booleanas.
  - Manipulação de Dados: Criar novas colunas e tratar dados faltantes (`NaN`) de forma eficiente.
  - **Agrupamento** (`groupby`): Sumarizar e agregar informações categóricas do dataset.
  - Exercícios Práticos: Aplicar essas técnicas em exercícios práticos.
  - Disponibilização da segunda lista de exercícios.
- **Aula 20:**
    - Exercícios realizados em aula.
  - **Aula 21:**
    - Entrega da segunda lista de exercícios.
    - Resolução da segunda lista de exercícios.

## Avaliação

A avaliação de rendimento nesta disciplina constará de três etapas distintas e independentes, compreendendo a realização de uma prova escrita individual (PI), sem qualquer tipo de consulta e realizada de forma estritamente presencial, além da entrega de duas listas (L1 e L2) de exercícios práticos, as quais também deverão ser desenvolvidas e submetidas de maneira estritamente individual. Cada um desses três instrumentos avaliativos será mensurado em uma escala de 0 (zero) a 100 (cem) pontos, de modo que a média final do discente será calculada por meio da média aritmética simples obtida a partir das três notas atribuídas ao longo do período letivo.

$$MF = \frac{(PI + L1 + L2)}{3}$$

As referidas listas de exercícios serão disponibilizadas de maneira gradativa ao longo do período letivo, com os respectivos prazos limites para submissão a serem oportunamente estipulados e divulgados pelo docente. Sob a perspectiva de padronização metodológica, todos os trabalhos práticos deverão ser estruturados e entregues obrigatoriamente no formato de Jupyter Notebook, devendo o discente seguir de forma rigorosa o modelo de documento e as diretrizes de formatação previamente estabelecidos e fornecidos pelo professor. O aluno estará aprovado na disciplina se  $MF \geq 70,0$  pontos. Caso contrário, reprovado.